

DeliverEats Development Workflow Guide

1. Project Architecture Agreement (Before Coding)

Create one shared documentation space (Notion / GitHub Wiki / Google Docs).

Must include:

Database Schema

Core tables:

- users
- restaurants
- menu_categories
- menu_items
- item_variants
- orders
- order_items
- rider_locations
- reviews
- payouts
- order_histories

Rule: No one creates or changes database structure without team agreement.

2. Naming Conventions

Models (Singular PascalCase)

Examples:

- Order
- Restaurant
- MenuItem
- RiderLocation

Tables (Plural snake_case)

Examples:

- orders

- restaurants
- menu_items

Controllers

Pattern:

- OrderController
- RestaurantController
- RiderController
- PaymentController

Services

Pattern:

- OrderService
- DispatchService
- PaymentService
- SurgePricingService

Events

Pattern:

- OrderPlaced
- OrderAssigned
- OrderDelivered
- RiderLocationUpdated

Jobs

Pattern:

- AssignRiderJob
- SendOrderNotificationJob
- ProcessPaymentJob

3. API Contract Rules

Every API must follow unified response format.

Success Response

```
{  
  "success": true,
```

```
"message": "",  
"data": {}  
}
```

Error Response

```
{  
  "success": false,  
  "message": "",  
  "errors": []  
}
```

Rule: Do not invent random response structures.

4. Route Structure

Customer Routes

- /api/customer/orders
- /api/customer/cart

Restaurant Routes

- /api/restaurant/menu
- /api/restaurant/orders

Rider Routes

- /api/rider/orders
- /api/rider/location

Admin Routes

- /api/admin/dashboard

Rule: Use RESTful naming only.

Examples:

- GET /orders
 - POST /orders
 - PUT /orders/{id}
 - DELETE /orders/{id}
-

5. Git Workflow

Branches

- main
- develop
- feature/orders
- feature/dispatch
- feature/restaurant
- feature/rider

Workflow: feature → develop → main

Rules:

- Never push directly to main
 - Pull develop before coding
 - Use Pull Requests before merging
-

6. Team Ownership Boundaries

Kareem

Owns:

- Core Architecture
- Orders
- Payments
- Surge Pricing
- Authentication

Sarhan

Owns:

- Dispatch System
- GPS Tracking
- Real-Time Events
- Redis Queues

Hanaa

Owns:

- Restaurant Management

- Reviews
- Restaurant Dashboard

Basmala

Owns:

- Rider Portal
- Notifications
- Testing
- Documentation

Rule: Do not modify another member's module directly.

7. Shared Constants Rule

Use constants for statuses.

Example:

- placed
- confirmed
- preparing
- on_the_way
- delivered
- cancelled

Rule: No hardcoded strings.

8. Weekly Integration Plan

Integration checklist:

- API routes working
 - Auth working
 - Database migrations valid
 - Events broadcasting correctly
 - Queue jobs running
-

9. Code Review Checklist

Before merging:

- Naming consistent?
 - Validation exists?
 - Auth middleware exists?
 - Route naming correct?
 - Tests written?
 - Response format correct?
-

10. Project Board Workflow

Using GitHub Projects.

Columns:

- Backlog
- Todo
- In Progress
- Review
- Done
- Blocked

Rule: Every task must exist on board.

11. Shared Seeders

Create test data for:

- Restaurants
- Riders
- Customers
- Orders

Purpose: Ensure all developers test on the same environment.

12. Daily Team Sync

Every day (10 minutes):

Each member answers: 1. What did I finish? 2. What am I building now? 3. What is blocking me?

Purpose: Prevent conflicts and misalignment.

Start Plan (Day 1)

Kareem

- Database schema
- Authentication
- Core API setup

Sarhan

- Redis setup
- WebSocket setup

Hanaa

- Restaurant CRUD

Basmala

- Testing setup
- Documentation setup

Day 2

Start integrations between modules.

Golden Rules

1. Shared naming conventions first.
2. Shared API contract first.
3. Branch properly.
4. Respect ownership boundaries.
5. Review before merge.
6. Integrate frequently.
7. Communicate daily.